

Experiment Project Design: Evaluating Cache Policy Adaptability to Workload Phase Shifts

Erik B. Ebbestad

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

erebb5820@uit.no

Ebbisso@github

Frode Eggenfellner

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

fregg8888@uit.no

Frodhesten@github

Martin Rasmussen

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

maras7515@uit.no

Martinstudererinformatick@github

I. OVERVIEW AND INTEGRATION WITH ASSIGNMENT 1

This project builds on Assignment 1 by looking at how different cache replacement policies handle sudden changes in memory access patterns. Most modern applications run in distinct phases, but classic policies like LRU, LFU, and FIFO use fixed rules to decide what to evict. We will test how quickly both standard and adaptive policies recover from spikes in cache misses after a workload changes phase.

This document outlines our experimental design, including the hardware, software, workloads, and performance metrics. It also details the exact steps we will take to keep our results reliable, isolated, and statistically sound.

II. HARDWARE ENVIRONMENT

Because the core of our methodology relies on a custom software-based cache simulator, the physical hardware environment dictates the simulation's runtime performance rather than the architectural properties of the cache being tested. Strict control over the host environment is necessary to prevent simulator bottlenecks and ensure accurate tracking of simulation overhead.

The simulation campaigns will be executed locally on an ASUS ZenBook S16 OLED (3K Touch) laptop. While this represents a consumer-grade mobile workstation, its modern architecture provides high computational throughput and fast I/O for our trace-driven simulations. The specifications are:

- **CPU:** AMD Ryzen™ AI 9 HX 370 (12 cores, 24 threads, base clock 2.0 GHz, boost up to 5.1 GHz).
- **Memory:** 32 GB LPDDR5X RAM. This capacity dictates that we may need to process larger trace files in chunks rather than loading them entirely into memory at once.
- **Storage:** 1 TB PCIe® 4.0 NVMe™ M.2 SSD. High sequential read throughput is required to stream large trace datasets into the simulator without heavy I/O blocking.
- **Operating System:** Windows 11 utilizing Windows Subsystem for Linux (WSL 2) running Ubuntu 24.04 LTS. This setup provides a stable, native Unix environment for script orchestration and compiling C-based simulations.

III. SOFTWARE ENVIRONMENT

The experimental campaign relies on a mix of custom simulation software, trace generation utilities, and data analysis scripts.

A. Custom Cache Simulator

A deterministic, trace-driven cache simulator will be developed in C. We opt for a custom simulator rather than existing tools (like gem5 or Dinero IV) to retain complete control over the telemetry gathered during the immediate transient window of a phase shift. The simulator uses a modular architecture where the cache state tracking and the replacement policy logic are strictly decoupled.

B. Supporting Tools

- **Compiler:** A standard C compiler (e.g., GCC or Clang) utilizing appropriate optimization flags for execution speed and debugging flags for validation.
- **Trace Generation:** Dynamic binary instrumentation frameworks or memory profiling tools (such as Valgrind or Intel PIN) will be evaluated and utilized to extract memory access traces from compiled benchmark applications if synthetic generation proves insufficient.
- **Data Analysis & Plotting:** Python along with standard data manipulation and visualization libraries (e.g., pandas, matplotlib, or seaborn) to aggregate simulation outputs and generate time-series plots of transient hit-rate recovery.
- **Automation:** Build automation utilities and scripting interfaces (e.g., Make, Bash, or Python scripts) to orchestrate parametric sweeps across different simulator configurations.

IV. WORKLOADS AND DATASETS

To isolate the specific behavior of cache policies during phase transitions, our primary datasets will be synthetically generated trace files, supplemented by annotated real-world traces.

A. Synthetic Workloads (Primary)

Synthetic workloads allow precise control over spatial and temporal locality. A Python-based trace generator will output files where each line represents a memory access: [Instruction/Data] [Hexadecimal Address] [Size]. We define three fundamental access behaviors:

- **Scanning (S):** Sequential memory accesses across a contiguous memory block significantly larger than the cache capacity.
- **Working Set (W):** Repeated random accesses within a tightly bounded memory region (smaller than the cache capacity) demonstrating high temporal locality.
- **Thrashing (T):** Random accesses across a massive address space with zero temporal or spatial locality.

Phases will be concatenated to form datasets representing specific transitions. For example, a dataset labeled W-S-W will contain multiple accesses of a small working set, followed abruptly by a sequence of scanning accesses, returning to the original working set.

B. Real-World Datasets (Secondary)

If synthetic traces fail to trigger realistic eviction thresholds, we will utilize memory traces from the SPEC CPU2017 benchmark suite. Using phase-analysis tools (like SimPoint), we will extract snippets of execution where the benchmark transitions between functionally distinct algorithmic phases (e.g., initialization phase $\rightarrow$ computation phase).

V. MODELS

Our custom simulator models a parametric set-associative cache hierarchy. Since we are simulating caching behavior offline via traces, we model structural parameters rather than exact cycle-level electrical timings.

The modeled parameters are designed to be flexible, allowing us to evaluate policies under various architectural constraints:

- **Cache Capacity (C):** Configurable range, scalable to represent anything from a small L1 cache to a larger L3 shared cache.
- **Block/Line Size (B):** Configurable, defaulting to standard architectural block sizes (e.g., 64 bytes).
- **Associativity (A):** Configurable from Direct-Mapped to Fully Associative, allowing us to test policies under different degrees of conflict constraints.

Timing models are strictly deterministic. We will define configurable hit and miss penalties to represent different memory hierarchy levels, allowing us to compute relative average memory access times based on eviction outcomes.

VI. EVALUATION AND PERFORMANCE METRICS

To answer our research questions, standard aggregate metrics are insufficient. We require both global and transient (windowed) metrics.

A. Global Metrics

- **Global Hit Rate (GHR):** The total number of cache hits divided by total accesses across the entire simulation.
- **Average Memory Access Time (AMAT):** Calculated as $Hit_Time + (Miss_Rate \times Miss_Penalty)$.

B. Transient/Phase Metrics

- **Windowed Miss Rate ($WMR_{\Delta t}$):** The miss rate computed over a sliding window of Δt memory accesses. This metric visualizes the spike in misses precisely when a phase transition occurs.
- **Recovery Latency (RL):** Following a phase shift, the number of memory accesses required for the WMR to stabilize within a specified threshold of its steady-state asymptotic limit for the new phase. A smaller RL indicates a faster-adapting replacement policy.
- **Pollution Ratio:** The percentage of cache lines occupied by "dead" blocks (data from the previous workload phase that will never be requested again) measured at varying intervals post-transition.

VII. EXPERIMENTAL PROTOCOLS

To systematically investigate policy behavior, we define three distinct experimental protocols. All experiments employ a warmup phase (filling the cache to capacity) before metric collection begins to eliminate cold-start bias.

A. Experiment 1: Baseline Policy Adaptability

Objective: Answer RQ1 and RQ2 by measuring recovery latency across standard policies.

- **Factors:** Replacement Policy (LRU, FIFO, LFU, ARC).
- **Parameters:** Baseline cache sizes and associativities, with a dynamically scaled observation window size based on pilot tests.
- **Workload:** Synthetic $W1 \rightarrow W2$ (Abrupt transition between two distinct, non-overlapping working sets appropriately scaled to the cache capacity).
- **Protocol:** Run trace through the simulator. Record WMR at every step. Identify the peak miss rate immediately post-transition and calculate RL for each policy. Repeat with multiple random seeds for the Working Set generation to ensure statistical significance.

B. Experiment 2: Resistance to Scan Thrashing

Objective: Evaluate how policies protect the primary working set during a brief, scan-heavy workload phase (e.g., an antivirus background scan or garbage collection).

- **Factors:** Replacement Policy.
- **Parameters:** Scaled cache configurations to simulate mid-tier cache levels where scan pollution is highly detrimental.
- **Workload:** $W \rightarrow S \rightarrow W$. The Scanning (S) phase size will be swept relative to the configured cache capacity (e.g., from a fraction of the cache size to multiple times its capacity).

- **Protocol:** Measure the WMR when the trace returns to phase W . Determine which policies successfully preserved the W data versus those that succumbed to frequency/recency pollution caused by S .

C. Experiment 3: Frequency of Transitions

Objective: Answer RQ3 by measuring the degradation of hit rates under high-frequency phase toggling.

- **Factors:** Toggle frequency (Length of stable execution between shifts).
- **Parameters:** Phase duration swept across a wide range of access counts to simulate both rapid and gradual shifts.
- **Workload:** $W1 \leftrightarrow W2$ alternating continuously.
- **Protocol:** Compare the GHR of adaptive policies (ARC) vs. static policies (LRU) as the frequency of shifts increases. At what toggle frequency does the overhead of adaptability outweigh its benefits?

VIII. TIMELINE

The remaining weeks of the semester will be structured into several sequential stages to ensure the experiments are executed systematically. First, we will focus on implementing the custom, trace-driven cache simulator in C, ensuring that the core data structures and memory telemetry logic are robust. Once the base simulator is operational, we will integrate the standard replacement policies (LRU, FIFO, LFU) and validate their eviction correctness using small, manually verifiable traces.

Following the validation of the basic simulator, we will develop the Python-based synthetic trace generators to create the controlled scanning, working set, and thrashing workloads. With the workloads ready, we will implement the advanced or adaptive policy (such as ARC or SIEVE) to serve as our primary comparison point. Pilot runs will then be conducted to fine-tune simulation parameters, particularly focusing on optimizing the sliding window size for transient miss-rate calculations.

With the environment fully calibrated, we will systematically execute the three main experimental protocols. We will begin with baseline adaptability and scan thrashing resistance tests, adjusting configuration parameters if early telemetry indicates anomalous behaviour. Afterward, we will conduct the high-frequency transition experiments.

Finally, the raw data extracted from the simulator will be parsed and analysed using Python. We will generate time-series plots and perform statistical analyses to evaluate recovery latency and pollution ratios. The final weeks will be dedicated to drafting the final project report, where we will critically discuss our findings, address any experimental limitations, and finalize the delivery.

IX. DISCUSSION AND DATA MANAGEMENT

A. Data Management

All source code, including the simulator, trace generators, and analysis scripts, will be hosted on a private GitHub repository shared among the group members and accessible

to the teaching assistants. Trace datasets will be managed systematically. Where synthetic traces are used, deterministic seeds will be stored in the repository, allowing any user to regenerate identical workloads on the fly. Experimental results and metrics will be exported strictly as CSV files to facilitate straightforward parsing and plotting with Python.

B. Risks and Confounders

Several risks have been identified in this design:

- 1) **Simulator Correctness:** An error in the LFU or ARC implementation could skew results. *Mitigation:* We will write unit tests for the eviction logic of single sets before scaling to full N-way caches.
- 2) **Unrepresentative Workloads:** Synthetic traces may be overly hostile, making standard policies look artificially poor. *Mitigation:* We include the secondary option of SPEC CPU2017 traces to anchor our findings in real-world application behavior.
- 3) **Window Size Artifacts:** The sliding window size (Δt) used for calculating transient miss rates acts as a low-pass filter. If Δt is too large, the transient spike is smoothed out; if too small, the data becomes noisy. *Mitigation:* We will perform an initial sensitivity analysis on the window size during pilot runs to find the optimal signal-to-noise ratio before running the final protocols.

X. USE OF AI

Artificial intelligence was used during the writing of this report to proofread the text and correct spelling, grammar, and phrasing.